

Reviewer Pack — Multilinear Catalecticant Rank Scales Polynomially in ACC^0 ...

Entry fec763a69d10 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 03:10:16 UTC

Multilinear Catalecticant Rank Scales Polynomially in ACC^0 Circuit Size

Entry ID: fec763a69d10 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 03:10:16 UTC

1. Conjecture

Field A (mathematical branch): Apolarity / classical invariant theory (Sylvester catalecticant matrices and partial-derivative ranks of multilinear polynomials) **Field B** (complexity object): $\text{ACC}^0[m]$ Boolean circuit size and depth (Williams-Beigel-Tarui $\text{SYM} \circ \text{AND}$ decomposition framework, target $\text{NEXP} \not\subseteq \text{ACC}^0$)

Statement:

For a Boolean function $f: \{0,1\}^n \rightarrow \{0,1\}$, let $\alpha_U \in \text{GF}(2)$ be its algebraic-normal-form (ANF) coefficient at $U \subseteq [n]$, and define the level- k multilinear catalecticant matrix $M^{\{k\}}(f) \in \text{GF}(2)^{C(n,k) \times C(n,k)}$ by $M^{\{k\}}(f)_{S,T} = \alpha_{S \cup T}$ if $S \cap T = \emptyset$ else 0; let $\text{MCR}_k(f)$ be its $\text{GF}(2)$ -rank. Conjecture: for every f computable by an $\text{ACC}^0[m]$ circuit of size s , depth d , and modulus $m \leq 6$, $\text{MCR}_{\lfloor n/2 \rfloor}(f) \leq (s+1)^{2d}$; whereas a uniformly random Boolean f satisfies $\text{MCR}_{\lfloor n/2 \rfloor}(f) \geq \frac{1}{2} \cdot C(n, \lfloor n/2 \rfloor)$ with probability $\geq 1 - 2^{-n}$. A single $\text{ACC}^0[m]$ circuit (s,d) whose MLE has $\text{MCR}_{\lfloor n/2 \rfloor} > (s+1)^{2d}$ falsifies it.

Rationale (proposer's reasoning):

Williams' route to $\text{NEXP} \not\subseteq \text{ACC}^0$ hinges on Beigel-Tarui's representation of ACC^0 functions as low-rank sums of simple monomial terms, and the catalecticant / partial-derivative rank is the canonical apolarity measure of exactly such low-rank decompositions of polynomials — yet apolarity has been used almost exclusively for arithmetic circuit lower bounds (Landsberg, Teitler, Shpilka-Wigderson), not for Boolean ACC^0 over $\text{GF}(2)$. If the bound holds, the same SAT-style derandomization that powers Williams' NEXP separation gains a concrete algebraic certificate; if it fails, we obtain an explicit ACC^0 circuit whose MLE has anomalously large catalecticant rank, which itself is informative. The mapping is purely computational (Möbius transform + $\text{GF}(2)$ rank) so it sidesteps the relativization barrier and is not a naturalization of any pseudo-random-function-defeating property because the bound is one-sided on circuits, not on truth-tables.

Taxonomy category: $\text{ACC_LB_via_WILLIAMS}$ (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c1bb3410e281669c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across $n \in \{8, 10, 12\}$ with 30 seeds each, every $\text{ACC}^0[6]$ circuit sample ($s \in \{n, 3n, 9n\}$, $d \in \{2, 3\}$) must satisfy $\text{MCR}_{\lfloor n/2 \rfloor} \leq (s+1)^{\{2d\}}; \geq 27/30$ random truth tables per n must satisfy $\text{MCR} \geq \frac{1}{2} \cdot C(n, \lfloor n/2 \rfloor)$; and the OLS slope of $\log \text{MCR}$ vs $\log s$ on ACC^0 samples must be $\leq 2d+0.5$ for each n .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	UNCERTAIN	0.90	HITS	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - catalecticant rank Boolean function ANF lower bound circuit complexity - ACC^0 partial derivative matrix rank multilinear polynomial $\text{GF}(2)$ - apolarity catalecticant symmetric circuit lower bound NEXP ACC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_random_circuit(n, s, d):
    gates = []
    for _ in range(s):
        gate_type = random.choice(['AND', 'OR', 'NOT', 'MOD_2', 'MOD_3'])
        if gate_type == 'NOT':
            inputs = [random.randint(0, n-1)]
        else:
            inputs = sorted(random.sample(range(n), 2))
        gates.append((gate_type, inputs))
    return gates

def evaluate_circuit(circuit, input_values):
    for gate_type, inputs in circuit:
        if gate_type == 'AND':
            result = all(input_values[i] for i in inputs)
        elif gate_type == 'OR':
```

```

        result = any(input_values[i] for i in inputs)
    elif gate_type == 'NOT':
        result = not input_values[inputs[0]]
    elif gate_type == 'MOD_2':
        result = sum(input_values[i] for i in inputs) % 2
    elif gate_type == 'MOD_3':
        result = sum(input_values[i] for i in inputs) % 3
    else:
        raise ValueError(f"Unknown gate type: {gate_type}")
    input_values.append(result)
return input_values[-1]

def compute_anf(truth_table):
    n = int(math.log2(len(truth_table)))
    anf = [0] * (1 << n)
    for i in range(1 << n):
        anf[i] = truth_table[i]
    for k in range(n-1, -1, -1):
        mask = 1 << k
        for i in range((1 << n) - 1, -1, -1):
            if i & mask:
                anf[i] ^= anf[i ^ mask]
    return anf

def compute_catalecticant_matrix(anf, n, k):
    from itertools import combinations
    C = list(combinations(range(n), k))
    M = [[0] * len(C) for _ in range(len(C))]
    for i, S in enumerate(C):
        for j, T in enumerate(C):
            if not set(S).intersection(set(T)):
                U = tuple(sorted(S + T))
                M[i][j] = anf[sum(1 << bit for bit in U)]
    return M

def gaussian_elimination(M):
    n = len(M)
    rank = 0
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(M[j][i]) > abs(M[max_row][i]):
                max_row = j
        M[i], M[max_row] = M[max_row], M[i]
        if M[i][i] == 0:
            continue
        rank += 1
        for j in range(i+1, n):
            factor = M[j][i] / M[i][i]
            for k in range(n):
                M[j][k] -= factor * M[i][k]
    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    results = []

```

```

for n in [8, 10, 12]:
    for s in [n, 3*n, 9*n]:
        for d in [2, 3]:
            circuit = generate_random_circuit(n, s, d)
            input_values = [random.randint(0, 1) for _ in range(n)]
            truth_table = [evaluate_circuit(circuit, input_values[:i] + [val] + input_values[i:]) for i, val in enumerate(input_values)]
            anf = compute_anf(truth_table)
            M = compute_catalecticant_matrix(anf, n, n // 2)
            rank = gaussian_elimination(M)
            results.append((s, d, rank))

random_truth_table = [random.randint(0, 1) for _ in range(1 << n)]
anf_random = compute_anf(random_truth_table)
M_random = compute_catalecticant_matrix(anf_random, n, n // 2)
rank_random = gaussian_elimination(M_random)

MAJ_n_truth_table = [int(i == n // 2) for i in range(1 << n)]
anf_MAJ_n = compute_anf(MAJ_n_truth_table)
M_MAJ_n = compute_catalecticant_matrix(anf_MAJ_n, n, n // 2)
rank_MAJ_n = gaussian_elimination(M_MAJ_n)

mean_rank = sum(rank for _, _, rank in results) / len(results)
std_rank = math.sqrt(sum((rank - mean_rank) ** 2 for _, _, rank in results) / len(results))
slope, intercept = 0, 0
if len(results) > 1:
    x = [math.log(s * d + 1) for s, d, _ in results]
    y = [math.log(rank) for _, _, rank in results]
    n = len(x)
    sum_x = sum(x)
    sum_y = sum(y)
    sum_xy = sum(xi * yi for xi, yi in zip(x, y))
    sum_xx = sum(xi ** 2 for xi in x)
    slope = (n * sum_xy - sum_x * sum_y) / (n * sum_xx - sum_x ** 2)

conjecture_holds = all(rank <= (s + 1) ** (2 * d) for s, d, rank in results)
random_support = sum(1 for rank in [rank_random, rank_MAJ_n] if rank >= Fraction(n, 2) * math.log(2))
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "MCR",
    "metric_value": mean_rank,
    "instances_tested": len(results),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i + 3**j + 5**k for i, j, k in range(3)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank = sum(res["metric_value"] for res in results) / len(results)

```

```

std_rank = math.sqrt(sum((res["metric_value"] - mean_rank) ** 2 for res in results) / len(results))
support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

if all(res["conjecture_holds"] for res in results):
    print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fraction}")
else:
    first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_eae0541d.py", line 141, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_eae0541d.py", line 95, in run_trial
    truth_table = [evaluate_circuit(circuit, input_values[:i] + [val] + input_values[i+1:]) for i in range(n)]
                  ^^^^^
NameError: name 'val' is not defined. Did you mean: 'eval'?

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a NameError ('val' is not defined') before any trials produced data, so neither the support nor falsification conditions could be evaluated. | next: Fix the truth-table construction bug in run_trial (properly bind 'val' when iterating over input assignments) and re-run the pre-registered protocol across $n \in \{8, 10, 12\}$.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	172897
2	preregistration	claude_max	opus	0	0	8803
3	novelty	claude_max	opus	0	0	5023
4	novelty	claude_max	opus	0	0	13842
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	58416

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	199065
7	judge	claude_max	opus	0	0	3961

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 462008 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in research/audit/fec763a69d10.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/fec763a69d10.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/fec763a69d10.tar.gz (if generated)